

ASSIGNMENT-1

1. Develop a Menu driven program to demonstrate the following operations of Arrays ——
MENU—— 1. CREATE 2. DISPLAY 3. INSERT 4. DELETE 5. LINEAR SEARCH 6. EXIT

Code:

```
// jayant mahajan
```

```
#include <stdio.h>
```

```
int arr[100];
```

```
int n = 0;
```

```
void create()
```

```
{
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements:
```

```
");
```

```
    for(int i=0;i<n;i++)
```

```
        scanf("%d",&arr[i]);
```

```
}
```

```
void display()
```

```
{
```

```
    printf("Array elements are: ");
```

```
    for(int i=0;i<n;i++)
```

```
        printf("%d ",arr[i]);
```

```
}
```

```
int main()
```

```

{
    int choice;
    do
    {
        printf("\n-----MENU-----\n");
        printf("1. CREATE\n2. DISPLAY\n3. EXIT\n");
        scanf("%d",&choice);

        switch(choice)
        {
            case 1: create(); break;
            case 2: display(); break;
        }
    }while(choice!=3);

    return 0;
}

```

Output:

```
-----MENU-----
```

```
1. CREATE
```

```
2. DISPLAY
```

```
3. EXIT
```

```
Enter choice: 1
```

```
Enter number of elements: 5
```

```
Enter elements:
```

```
10 20 30 40 50
```

```
Enter choice: 2
```

Array elements are: 10 20 30 40 50

2. Design the logic to remove the duplicate elements from an Array and after the deletion the array should contain the unique elements.

Code:

```
#include <stdio.h>
// jayant mahajan
int main()
{
    int arr[100], n, i, j, k;

    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);

    for(i=0;i<n;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(arr[i]==arr[j])
            {
                for(k=j;k<n-1;k++)
                    arr[k]=arr[k+1];

                n--;
                j--;
            }
        }
    }
```

```

    }
}

for(i=0;i<n;i++)
    printf("%d ",arr[i]);

return 0;
}

```

Output:

Input:

10 20 30 20 40 10

Output:

10 20 30 40

3. Design the logic of a program to rotate a 1-D array by K positions. e.g., for A = {1,2,3,4,5} and K = 2, the left rotation is {3,4,5,1,2} and the right rotation is {4,5,1,2,3}.

Code:

```

#include <stdio.h>

// jayant mahajan

int main()
{
    int arr[]={1,2,3,4,5};
    int k=2;
    int i;

```

```

printf("Left Rotation: ");
for(i=k;i<5;i++)
    printf("%d ",arr[i]);

for(i=0;i<k;i++)
    printf("%d ",arr[i]);

printf("\nRight Rotation: ");

for(i=5-k;i<5;i++)
    printf("%d ",arr[i]);

for(i=0;i<5-k;i++)
    printf("%d ",arr[i]);

return 0;
}

```

Output:

Left Rotation: 3 4 5 1 2

Right Rotation: 4 5 1 2 3

4. Implement the logic to
 - a. Reverse the elements of an array
 - b. Find the matrix multiplication
 - c. Find the Transpose of a Matrix

Code:

- a) Reverse the elements of an array:

```

#include <stdio.h>
// jayant mahajan
int main()
{
    int arr[100], n, i, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:
");
    for(i=0; i<n; i++)
        scanf("%d", &arr[i]);

    for(i=0; i<n/2; i++)
    {
        temp = arr[i];
        arr[i] = arr[n-i-1];
        arr[n-i-1] = temp;
    }

    printf("Reversed array:
");
    for(i=0; i<n; i++)
        printf("%d ", arr[i]);

    return 0;
}

```

Output:

Input:

1 2 3 4 5

Output:

5 4 3 2 1

b) Matrix Multiplication:

```

#include <stdio.h>
// jayant mahajan

```

```
int main()
{
    int a[10][10], b[10][10], c[10][10];
    int r1, c1, r2, c2;
    int i, j, k;

    printf("Enter rows and columns of first matrix: ");
    scanf("%d%d", &r1, &c1);

    printf("Enter rows and columns of second matrix: ");
    scanf("%d%d", &r2, &c2);

    if(c1 != r2)
    {
        printf("Matrix multiplication not possible");
        return 0;
    }

    printf("Enter first matrix elements:
");
    for(i=0;i<r1;i++)
        for(j=0;j<c1;j++)
            scanf("%d",&a[i][j]);

    printf("Enter second matrix elements:
");
    for(i=0;i<r2;i++)
        for(j=0;j<c2;j++)
            scanf("%d",&b[i][j]);

    for(i=0;i<r1;i++)
    {
        for(j=0;j<c2;j++)
        {
            c[i][j]=0;
            for(k=0;k<c1;k++)
                c[i][j]+=a[i][k]*b[k][j];
        }
    }

    printf("Resultant Matrix:
");
```

```

    for(i=0;i<r1;i++)
    {
        for(j=0;j<c2;j++)
            printf("%d ",c[i][j]);

        printf("\n");
    }

    return 0;
}

```

Output:

Resultant Matrix:

19 22

43 50

c) Transpose of a Matrix:

```

#include <stdio.h>
// jayant mahajan
int main()
{
    int a[10][10], transpose[10][10];
    int r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d%d",&r,&c);

    printf("Enter matrix elements: ");

    for(i=0;i<r;i++)
        for(j=0;j<c;j++)
            scanf("%d",&a[i][j]);

    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            transpose[j][i]=a[i][j];
        }
    }
}

```

```

    }

    printf("Transpose of Matrix:
");

    for(i=0;i<c;i++)
    {
        for(j=0;j<r;j++)
            printf("%d ",transpose[i][j]);

        printf("\n");
    }

    return 0;
}

```

Output:

Original Matrix:

```

1 2 3
4 5 6

```

Transpose Matrix:

```

1 4
2 5
3 6

```

5. Write a program to find sum of every row and every column in a two-dimensional array.

Code:

```

#include <stdio.h>

// jayant mahajan

int main()
{
    int a[10][10],r,c,i,j;

```

```

scanf("%d%d",&r,&c);

for(i=0;i<r;i++)
    for(j=0;j<c;j++)
        scanf("%d",&a[i][j]);

for(i=0;i<r;i++)
{
    int sum=0;
    for(j=0;j<c;j++)
        sum+=a[i][j];

    printf("Row %d sum = %d\n",i+1,sum);
}

for(j=0;j<c;j++)
{
    int sum=0;
    for(i=0;i<r;i++)
        sum+=a[i][j];

    printf("Column %d sum = %d\n",j+1,sum);
}

return 0;
}

```

Output:

Row 1 sum = 6

Row 2 sum = 15

Row 3 sum = 24

Column 1 sum = 12

Column 2 sum = 15

Column 3 sum = 18